

Projet d'Intelligence Artificielle Développement d'un agent pour le jeu Connect 5

1 Objectif du projet

L'objectif de ce projet est de concevoir et d'implémenter une intelligence artificielle capable de jouer au jeu **Connect 5**. Chaque groupe devra développer un agent autonome capable d'analyser l'état courant du plateau et de décider du meilleur coup à jouer.

Ce projet permet de mettre en pratique plusieurs notions importantes en intelligence artificielle, notamment :

- la modélisation d'un problème de jeu ;
- la représentation d'états ;
- la prise de décision automatique ;
- la conception d'heuristiques ;
- l'évaluation expérimentale d'un agent ;

2 Présentation du jeu

Le jeu **Connect 5** est une variante du jeu de puissance. Il se joue à deux joueurs sur une grille rectangulaire. À chaque tour, un joueur choisit une colonne valide, puis son pion tombe dans la case libre la plus basse de cette colonne.

L'objectif est d'être le premier à aligner **cinq pions consécutifs** :

- horizontalement ;
- verticalement ;
- en diagonale montante ;
- en diagonale descendante.

Si le plateau est rempli sans qu'aucun joueur n'obtienne un alignement de cinq pions, la partie est déclarée nulle.

3 Travail demandé

Chaque groupe doit développer une classe Python nommée `IntelligentPlayer` qui hérite de la classe `BasePlayer` définie dans le fichier `player.py`.

Le fichier remis par chaque groupe doit obligatoirement respecter le format suivant :

- nom du fichier : `groupXX.py`
- nom de la classe : `IntelligentPlayer`

où `XX` est un numéro allant de 01 à 16.

Par exemple :

- `group01.py`
- `group12.py`
- `group16.py`

3.1 Méthode à implémenter

Votre agent doit implémenter la méthode `choose_move(board)`. Cette méthode sera appelée automatiquement à chaque tour avec l'état courant du plateau.

Exemple de structure minimale :

```
from player import BasePlayer

class IntelligentPlayer(BasePlayer):
    def choose_move(self, board):
        # board contient l'état courant du jeu
        # La méthode doit retourner un coup valide
        return Move(0)
```

Le plateau `board` ainsi que les autres éléments techniques nécessaires sont définis dans le code fourni avec le projet. Vous devrez donc vous appuyer sur l'API existante pour inspecter l'état du jeu et choisir un coup valide.

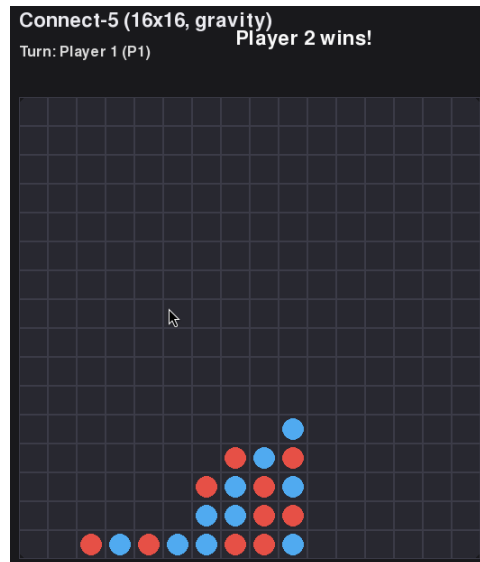
3.2 Tests

Vous pouvez tester le jeu avec la commande suivante :

```
python game.py --p1 player.HumanPlayer --p2 randomplayer.RandomPlayer
```

Ensuite, vous pourrez remplacer `randomplayer.RandomPlayer` par votre propre agent pour effectuer vos essais.

Interface de jeu :



4 Contraintes techniques

- Votre classe doit hériter de `BasePlayer`.
- Votre agent doit retourner un coup valide à chaque tour.
- La méthode `choose_move(board)` doit répondre dans un temps limité.
- Le temps maximal autorisé par tour est fixé à **1 seconde** à titre indicatif.
- Ce temps dépend du matériel, mais **tous les agents seront exécutés sur la même machine**, ce qui garantit l'équité lors de l'évaluation.
- En cas de dépassement du temps autorisé, l'agent est automatiquement disqualifié.
- En cas d'erreur d'exécution ou de coup invalide, la manche peut être considérée comme perdue.

4.1 Bibliothèques autorisées

Les étudiants peuvent utiliser :

- les modules de la bibliothèque standard Python ;
- les bibliothèques externes usuelles liées à l'algorithmique, au calcul scientifique ou au machine learning.

Par exemple, l'utilisation de bibliothèques comme `sklearn` est autorisée.

Il reste cependant de votre responsabilité de proposer une solution exécutable dans les contraintes de temps imposées. Une solution très lourde, même correcte sur le plan algorithmique, peut être pénalisée si elle est trop lente dans le cadre du tournoi.

5 Organisation des groupes

La promotion sera organisée comme suit :

- **12 groupes de deux étudiants ;**
- **4 étudiants en solo.**

Afin de préserver l'équité, les étudiants travaillant seuls bénéficieront de **+1 point**.

6 Évaluation

Le projet sera évalué selon les critères suivants :

1. Présentation orale : 8 points

Chaque groupe réalisera une présentation de **10 minutes**. Cette présentation devra expliquer clairement l'approche choisie, les idées principales de l'agent, les éventuelles heuristiques utilisées, ainsi que les choix techniques et expérimentaux.

2. Code source : 4 points

Le code sera évalué selon sa qualité, sa lisibilité, son organisation, sa robustesse, ainsi que la compréhension réelle du groupe de la solution proposée.

3. Rapport technique : 4 points

Un rapport de **2 pages minimum** est demandé. Il devra présenter le problème, l'approche choisie, les idées algorithmiques principales, les tests réalisés, les résultats obtenus, ainsi que les limites éventuelles de la solution.

4. Performance en tournoi : 4 points

Une partie de la note dépendra des performances de votre agent face aux autres agents lors du tournoi final.

7 Tournoi final

Les agents soumis par les groupes participeront à un tournoi final.

7.1 Principe

- Les affrontements seront organisés sous forme de **brackets déterminés aléatoirement**.
- Le gagnant de chaque affrontement avance au tour suivant.
- Chaque match comporte **4 manches**.
- Chaque agent commence **2 manches** afin de limiter l'avantage lié au premier coup.

7.2 Gestion des égalités

En cas d'égalité sur le nombre de manches gagnées :

- le **temps total d'exécution** des deux agents sera comparé ;
- l'agent le plus rapide sera déclaré vainqueur.

Ainsi, l'efficacité algorithmique est un critère important au même titre que la qualité stratégique.

7.3 Attribution des points

Le barème du tournoi est le suivant :

- **1 point** pour chaque match remporté ;
- le gagnant final du tournoi totalise donc **4 points**.

8 Livrables

Chaque groupe devra remettre :

- un fichier Python nommé `groupXX.py` contenant la classe `IntelligentPlayer` ;
- un rapport technique de minimum 2 pages ;
- une présentation orale de 10 minutes ;
- tout élément nécessaire à la compréhension de la solution, si demandé par l'enseignant.

9 Recommandations

- Testez votre agent contre plusieurs types d'adversaires.
- Vérifiez systématiquement que les coups retournés sont valides.
- Faites attention au temps de calcul.
- Concevez une solution claire, justifiable et robuste.
- Préparez une présentation structurée et pédagogique.